

Оглавление

Introduction	3
1 Обзор литературы	6
1.1 Вычислительный юмор как исследовательская проблема	6
1.2 Эволюция подходов к генерации юмора	7
1.3 Проблемы оценки юмора	8
1.4 Перспектива рассуждения для генерации юмора	9
1.5 Стандартное постобучение плохо подходит юмору	11
1.6 Постобучение, ориентированное на рассуждение	12
1.7 Исследовательский пробел	13
2 Методология	15
2.1 Границы исследования и статус реализации	15
2.2 Построение набора данных	16
2.3 Плотный индекс векторных представлений	16
2.4 Извлечение ключевых слов как поиск слабых запросов	17
2.5 Построение эталонных ответов через группировку ключевых слов и поиск	18
2.5.1 Группировка ключевых слов и формирование запросов .	18
2.5.2 Поиск эталонных ответов с помощью индекса векторных представлений	19
2.5.3 Удаление дублей и разбиение выборки	20
2.6 Генерация кандидатов	21
2.7 Оценка кандидатов	21
2.7.1 Попарное сравнение	21
2.7.2 Агрегация таблицы результатов	22
2.8 Теоретическая рамка обучения MRVF	22
2.8.1 Обозначения	23

2.8.2	RL с верифицируемыми наградами	24
2.8.3	VeriFree	26
2.8.4	С несколькими эталонными ответами VeriFree	28
2.8.5	Неполные наблюдаемые эталонные ответы	30
2.8.6	Предлагаемый цикл обучения	33
2.8.7	Замечание о использование базового уровня	35
3	Результаты	37
3.1	Текущие результаты	37
3.2	Планируемая процедура обучения	38
3.3	Планируемые абляции	39
	Список литературы	40

Введение

Генерация юмора является удобным способом изучения ограничений современных методов постобучения языковых моделей. В таких областях, как математика или программирование, сгенерированный ответ часто можно проверить внешней процедурой: решение либо совпадает с правильным ответом, либо программа проходит тесты или не проходит их. Юмор устроен иначе. Шутка может быть грамматически правильной, связной и соответствующей теме, но при этом не производить нужного эффекта. При этом проблема не сводится только к субъективности юмора. Один и тот же запрос может допускать множество разных удачных шуток, тогда как наблюдаемый набор данных обычно содержит лишь небольшую их часть. Поэтому юмор является не только сложной задачей генерации, но и полезным примером для изучения рассуждения и обучения с подкреплением в невалидируемых доменах [Loakman et al., 2025a, Lemmens and De Marez, 2026].

Эта сложность стала особенно важной после появления больших языковых моделей. Ранние системы вычислительного юмора были узкими, но интерпретируемыми: они опирались на шаблоны, лексические ресурсы или конкретные механизмы каламбуров [Ritchie, 2001, Amin and Burghardt, 2020]. Современные языковые модели обладают гораздо более широким покрытием, однако недавние работы показывают, что беглость текста не равна сильной юмористической компетентности. Крупные оценки по-прежнему помещают модельные системы ниже сильных человеческих результатов, а исследования каламбуров и понимания шуток показывают, что модели часто надежнее воспроизводят поверхностную форму юмора, чем его внутренний механизм [Zhang et al., 2024, Xu et al., 2024, Cocchieri et al., 2025, Loakman et al., 2025b]. Иными словами, юмор выявляет знакомую слабость современных моделей: они хорошо продолжают вероятный текст, но хуже выполняют открытый поиск по множеству правдоподобных и неожиданных продолжений.

Один из ответов на эту слабость состоит в явном моделировании про-

цесса рассуждения. Современные системы генерации юмора все чаще используют поэтапное построение подсказок, творческий поиск или декомпозицию на основе теории, а не одношаговое завершение текста. Эти подходы мотивированы простым наблюдением: хорошие шутки часто возникают не через выбор первого вероятного продолжения, а через исследование ассоциаций, переосмыслений, контрастов и форм панчлайна [Chen et al., 2023, Zhong et al., 2024, Tikhonov and Shtykovskiy, 2024, Kim and Chilton, 2025, Zhang et al., 2025]. Результаты выглядят многообещающе, но имеют ограничение. В большинстве таких систем структура рассуждения задается вручную: исследователь определяет число этапов, тип промежуточных представлений и порядок действий модели.

Параллельное направление в постобучении предлагает другую возможность. В верифицируемых доменах обучение с подкреплением недавно использовалось для извлечения развернутого поведения рассуждения напрямую, а не только через ручное построение подсказок или фиксированные процессы [Lightman et al., 2023, DeepSeek-AI, 2025]. Методы без внешнего верификатора, такие как VeriFree и RLPR, дополнительно показывают, что часть этой логики может быть перенесена за пределы математики и программирования, если заменить внешнего верификатора внутренними вероятностными целями [Zhou et al., 2025, Yu et al., 2025]. Однако для юмора такие формулировки сразу сталкиваются с несоответствием самой природе задачи. Они построены вокруг предположения об одной эталонной ссылке, тогда как юмор естественно является задачей с несколькими эталонными ответами: для запроса вроде “write a joke about a frog” существует много приемлемых ответов, и любой набор данных фиксирует только некоторые из них.

Данная работа мотивирована именно этим несоответствием. Ее центральное утверждение состоит в том, что генерацию юмора следует формулировать не как точную имитацию наблюдаемой шутки, а как распределение вероятностной массы по множеству приемлемых шуток при заданном запросе. Этот сдвиг важен и теоретически, и практически. Теоретически он меняет способ определения обучения без внешнего верификатора с подкреплением в творческом домене. Практически он меняет требования к данным: вместо изолированных пар запрос–ответ обучение требует наборов эталонных ответов, связанных с одним запросом и содержащих несколько правдоподобных

шуток.

Работа поэтому состоит из двух связанных частей. Первая часть является методологической и теоретической: в ней формулируется расширение обучения с подкреплением без внешнего верификатора на случай нескольких эталонных ответов для генерации юмора и анализируются последствия неполноты наблюдаемых эталонных наборов. Вторая часть является реализационной: она строит процесс данных, необходимый для практического применения такой цели. В текущем репозитории этот процесс уже включает сбор корпуса, генерацию плотных векторных представлений, извлечение ключевых слов, построение запросы, поиск эталонных ответов, генерацию кандидатов и автоматическую оценку. Отсутствующим этапом остается сам цикл обучения. Соответственно, данную работу следует читать как предварительное исследование в точном смысле этого термина: она задает формальную цель и программную инфраструктуру, необходимую для последующего обучающего эксперимента.

Источником данных является объединенный корпус примерно из 664k шуток, собранный из коллекций Short Jokes и rJokes. Полный корпус используется как пул поиска, тогда как его ключевых слов достаточно для построения обучающего набора данных с несколькими эталонными ответами. Целевая экспериментальная постановка является узким и контролируемым: модели Qwen3-1.7B и Qwen3-4B в режимах instruct и thinking будут сравниваться с вариантом thinking, обученным с помощью MRVF; обучение планируется на NVIDIA H200 GPU с использованием модифицированного trl и vLLM для генерации траекторий и вывода базовых моделей.

Глава 1

Обзор литературы

1.1 Вычислительный юмор как исследовательская проблема

Юмор давно рассматривается как сложный случай для обработки естественного языка, поскольку успешные шутки зависят не только от грамматической правильности. Они также зависят от несоответствия ожиданиям, общего знания, тайминга и ожиданий аудитории. Ранние обзоры уже описывали юмор как одну из наиболее сложных целей для искусственного интеллекта, отмечая, что реализованные системы в основном ограничивались узкими механизмами каламбуров, а более широкая обработка юмора потребовала бы значительного знания о мире и способности к рассуждению [Ritchie, 2001]. Два более поздних обзора приходят к сходному выводу с разных сторон. Amin and Burghardt [2020] показывают, что генерация вычислительного юмора исторически обменивала широту на контролируемость: шаблонные и основанный на правилах системы могли хорошо работать при наличии явного юмористического механизма, но только в сильно ограниченных доменах. Более недавно Loakman et al. [2025a] и Lemmens and De Marez [2026] утверждают, что эпоха больших языковых моделей расширила покрытие, но не решила центральную проблему, поскольку беглый текст не эквивалентен юмору, который читатели действительно находят смешным.

Дополнительная трудность связана с теоретическим плюрализмом. Теории юмора предлагают несколько повторяющихся классов объяснений—прежде всего подходы, основанные на несоответствии, но также традиции превосходство и разрядка,—и при этом ни одна рамка не исчерпывает феномен полностью. Это важно вычислительно, потому что разные типы шуток опираются на разные механизмы. Например, Kao et al. [2016] формализуют

несоответствие в каламбурах как неоднозначность плюс отличительность и показывают, что эти переменные предсказывают человеческие оценки смешного. Их работа важна не только тем, что находит применение классической теории юмора, но и тем, что показывает пределы узкой формализации: модель, объясняющая омофонические каламбуры, не начинает автоматически объяснять нарративные шутки, тематический шутки или социально ситуированный юмор. Современная основанный на теории работа по обнаружению юмора исходит из той же предпосылки: единой всеобъемлющей теории юмора недостаточно, а вычислительные модели обычно усваивают только фрагменты юмора [De Marez et al., 2024]. Для данной работы следствие является методологическим: юмор следует рассматривать как творческую задачу со множеством потенциальных механизмов, а не как один скалярный атрибут коммуникации.

1.2 Эволюция подходов к генерации юмора

До появления больших языковых моделей системы генерации юмора обычно были узкими по замыслу. Как резюмирует Amin and Burghardt [2020], ранние работы часто опирались на вручную написанные шаблоны, лексические ресурсы, фонетические преобразования или жестко ограниченные процедуры генерации. У этих систем было важное преимущество: они кодировали по крайней мере один явный юмористический механизм. Их недостатком была низкая адаптивность. Они генерировали юмор только при тщательно выбранных лингвистических условиях и поэтому плохо масштабировались за пределы каламбуров, устойчивых форм шуток или ограниченных доменов.

Большие языковые модели переворачивают этот компромисс. Они могут генерировать открытый юмористический текст по гораздо более широкому диапазону запросы, но беглость не гарантирует устойчивую юмористическую компетентность. Это видно в недавних крупномасштабных оценках. Zhang et al. [2024] вводят мультимодальный контрольный набор для подписей к карикатурам, построенный более чем на 250 миллионах человеческих оценок для более чем 2.2 миллиона подписи, и показывают, что современные методы дообучение остаются ограниченными на творческих задачах. Даже сильные системы, такие как GPT-4 и Claude, остаются ниже лучших чело-

веческих участников. Исследования понимания юмора указывают в том же направлении. Xu et al. [2024] показывают, что большие языковые модели демонстрируют то, что они называют lazy каламбур генерация: вместо слияния двух значений в один компактный каламбур модель часто отдельно называет оба связанных слова или значения, поверхностно удовлетворяя задаче. Этот результат методологически важен, поскольку показывает, что простые лексический успех метрики могут переоценивать способность к каламбурам. Оценка должна также отличать структурное качество и оригинальность от механически собранной или скопированной игры слов. Cocchieri et al. [2025] строят устойчивый к загрязнению данных контрольный набор юмора и показывают, что большинство языковых моделей все еще уступают людям в понимании, разрешении и генерации каламбуров. Выходя за пределы каламбуров, Loakman et al. [2025b] показывают, что современные модели, включая модели рассуждения, не объясняют надежно все типы шуток, что подчеркивает, насколько большая часть литературы по-прежнему сфокусирована на относительно простых формах юмора.

В совокупности эти исследования показывают, что современные модели лучше воспроизводят поверхностную форму знакомых шуток, чем обобщают юмористические механизмы. Это различие важно для данной работы. Система может хорошо генерировать текст, похожий на шутку, но не быть хорошей в юморе в более сильном смысле, необходимом для открытой генерации.

1.3 Проблемы оценки юмора

Оценка является центральной трудностью вычислительного юмора. Юмор субъективен, но проблема не только в субъективности в разговорном смысле. То, что считается хорошей шуткой, зависит от аудитории, контекста, подачи и оцениваемого измерения. Даже статьи о набора данных подчеркивают, что юмор меняется по теме, времени и получателю, из-за чего трудно выделить одну целевую переменную [Weller and Seppi, 2020]. Поэтому современные работы по юмору все чаще избегают трактовки смешность как скалярной метрики.

Наиболее ясный пример дает Sakabe et al. [2025], оценивающий Oogiri ответы по шести измерениям: новизна, ясность, релевантность, интеллект, эм-

патия и общая смешность. Их результаты показывают, что современные языковые модели генерируют ответы между низким и средним уровнем человеческих результатов, но заметно отстают по эмпатия. Авторы также утверждают, что этот дефицит помогает объяснить, почему модельный оценка плохо воспроизводит человеческую оценку юмора. Значение этого результата двоякое. Во-первых, он показывает, что качество юмора не сводится к вопросу “является this смешной?” Во-вторых, он показывает, что человеческие и модельные суждения могут отдавать приоритет разным качествам.

Дополнительная перспектива представлена у Winters and Van der Stockt [2025], которые сравнивают человеческих комиков и GPT-4 в живом импровизационном постановке, а не в офлайн только текстовый контрольный набор. Их исследование ценно тем, что задает более сильную точку сравнения: и люди, и модель должны быстро отвечать на одни и те же предложения аудитории. Человеческие шутки все еще предпочитают немного чаще, но разрыв не является подавляющим, а дизайн подчеркивает, насколько сильно подача и ситуативный контекст влияют на оценку юмора. Это полезная коррекция к мышлению только через контрольный набор. Текст, хорошо работающий в статической оценке, не обязательно является текстом, который лучше всего работает в реальном комедийном взаимодействии.

Эти результаты также уточняют роль методов помощью языковой модели-оценщика. Структурированные оценочные рамки, такие как у Hashemi et al. [2024], полезны, потому что заменяют одномерный оценивание явными рубриками и калибровкой. Однако эта работа также показывает, что сырые LLM рубрика суждения не автоматически хорошо согласуются с человеческими рейтингами. Для генерации юмора это означает, что LLM оценщики лучше рассматривать как измерительные инструменты с ограниченной областью применимости, а не как прозрачную замену человеческих предпочтений.

1.4 Перспектива рассуждения для генерации юмора

Один из наиболее важных сдвигов в современных исследованиях генерации юмора заключается в переходе от с одним примером продолжение к явно-

му рассуждению или поэтапному творческому поиску. На уровне построение подсказок Chen et al. [2023] моделируют генерацию юмора через пошаговые инструкции комедийного письма для GPT-3. Zhong et al. [2024] идут дальше и утверждают, что стандартный цепочки рассуждений построение подсказок плохо подходит творческим задачам; они предлагают парадигму Leap-Thought для Oogiri-style генерации юмора, где акцент делается на ассоциативных скачках, а не на строго последовательной декомпозиции.

Более явные системы генерации усиливают это направление. Tikhonov and Shtykovskiy [2024] реконструируют написание короткая шутка шутки как многошаговый процесс и сообщают об улучшениях относительно без примеров GPT-4 и других базовые модели в человеческой оценке. Kim and Chilton [2025] описывают генерацию юмора как комбинацию когнитивных, социальных и творческих навыков. В их исследовании теме-подписи к изображениям дополненный навыками системы существенно превосходят базовые модели на больших языковых моделях и приближаются к лучшим человеческим подписи для целевой аудитории. В мультимодальных условиях Zhang et al. [2025] предлагают направляемый теорией multi-этап рассуждение рамка, направленный на улучшение и интерпретируемости, и качества юмора.

Эти статьи важны, потому что коллективно отвергают предположение, что юмор лучше всего порождается мгновенным завершением. Вместо этого они моделируют генерацию юмора как процесс поиска по предпосылки, ассоциации, переосмысления и зависящий от аудитории варианты, аналогично тому, как человеческие эксперты подходят к комедийному письму или другим творческим задачам. Между предпосылка и шуткой может существовать информационный разрыв, который заполняется только во время инференс и может быть латентным. В то же время эти подходы разделяют ограничение, напрямую релевантное данной работе: структура рассуждения задается жестко. Число этапов, тип промежуточных представлений и порядок операций фиксируются исследователем. Поэтому такие системы являются сильной демонстрацией полезности рассуждения, но более слабым свидетельством того, что модель может сама научиться, когда и как рассуждать для юмора.

1.5 Стандартное постобучение плохо подходит юмору

Самый прямой путь к улучшению генерации юмора — с учителем дообучение на больших корпусах шуток. На практике, однако, такая целевая функция плохо соответствует структуре задачи. Набор данных шуток содержат множество повторяющихся завязки, знакомых паттернов игра слов и конвенциональных форм панчлайн. В результате с учителем дообучение может улучшать стилистическую правдоподобность, не обязательно улучшая творческое обобщение. Крупные ресурсы юмора, такие как rJokes, ценны масштабом и тематическим разнообразием, но они не были построены как с несколькими эталонными ответами генерация наборы данных, где много разных ответы одновременно могут быть правильными [Weller and Seppi, 2020].

Современная литература усиливает это опасение с двух сторон. Во-первых, юмор-specific работы показывают, что даже очень большие предпочтение наборы данных не делают творческий выравнивание простой задачей. Zhang et al. [2024] прямо подчеркивают ограничения RLHF- и DPO-style дообучение на творческом подписи к изображениям. Во-вторых, работы по после-обучение вне юмора делают различие запоминание–обобщение более резким. Chu et al. [2025] показывают, что обучение с подкреплением обобщает эффективнее, чем с учителем дообучение, на ненаблюдаемый текстовый и визуальный варианты, тогда как с учителем дообучение склонен запоминать обучение паттерны и испытывать трудности out распределение. Этот контраст особенно важен для генерации юмора, где слабый запрос может иметь много хороших продолжений, а запоминание частых оболочек шуток не равно созданию уместного и неожиданного панчлайн.

Preference оптимизация имеет собственные трудности. RLHF предполагает, что награда модель может выучить устойчивый приближенная мера для целевого поведения, но суждения о юморе шумны и неоднородны. Gao et al. [2023] в более общем виде показывают, что агрессивная оптимизация несовершенной награда модель может ухудшать истинный качество. Humor-specific оценка показывает, что риск усиливается в творческих доменах: Sakabe et al. [2025] показывают систематическое расхождение между человеческий и модель юмор суждения, а Winters and Van der Stockt [2025] показывают, что

качество юмора зависит от аспектов качества, которые трудно захватить офлайн текстовый награда сигналы. По этим причинам ни с учителем дообучение, ни стандартная предпочтение оптимизация не решают центральную проблему чисто.

1.6 Постобучение, ориентированное на рассуждение

Недавний успех ориентированный на рассуждение после-обучение в основном пришел из доменов с объективной верификацией. Wei et al. [2022] показывают, что промежуточное рассуждение может существенно менять поведение модели на задачах, требующих многошагового инференс. Lightman et al. [2023] расширяют эту логику от построение подсказок к разметка, показывая, что процесс разметка превосходит только по результату разметка на контрольный набор MATH. DeepSeek-AI [2025] затем демонстрируют, что обучение с подкреплением может вызывать длинный формат рассуждение без опоры на написанные людьми цепочки `thought`, причем наиболее ясные выигрыши наблюдаются в верифицируемых доменах, таких как математика, программирование и связанные STEM задачи.

Это создает центральную перенос проблема для юмора. Юмор также может выигрывать от расширенного рассуждения, но обычно не имеет детерминированного верификатора, способного объявить шутку правильной. Современные без внешнего верификатора подходы для доменов вне математики и программирования пытаются закрыть этот разрыв, заменяя явный проверяемую награду внутренними вероятностями, назначенными эталонные ответы. Zhou et al. [2025] предлагают VeriFree и показывают, что при с одним эталонным ответом предположение максимизация условной вероятности эталонный ответ при заданных запрос и траектория рассуждения эквивалентна максимизации точное совпадение верификатор целевая функция в ожидании. Yu et al. [2025] развивают близкий подход RLPR, где награда формулируется как среднее правдоподобие по отдельным токенам для повышения стабильности дисперсии.

Однако для юмора с одним эталонным ответом предположение не является малой технической деталью; это неверная модель данных. Prompts

вроде “write a joke about a frog” по природе допускают много правдоподобных ответов, и любой набор данных будет наблюдать только небольшое и смещенное их подмножество. Иными словами, юмор одновременно является с несколькими эталонными ответами и только частично наблюдаемый задачей. Это означает, что без внешнего верификатора рассуждение методы концептуально релевантны генерации юмора, но не применимы напрямую без переформулировки награда вокруг множеств эталонных ответов, а не одиночных целей.

1.7 Исследовательский пробел

Рассмотренная литература оставляет точный пробел. Исследования генерации юмора все чаще показывают, что поэтапный рассуждение, навык декомпозиция или направляемый теорией генерация могут улучшать ответы, но такие системы в основном опираются на жесткие инференс процессы [Chen et al., 2023, Zhong et al., 2024, Tikhonov and Shtykovskiy, 2024, Kim and Chilton, 2025, Zhang et al., 2025]. Работы по ориентированный на рассуждение обучение с подкреплением показывают, что полезные рассуждение поведение можно вызывать обучением, но сильнейшие существующие формулировки предполагают либо объективную верифицируемость, либо один эталонный ответ [Lightman et al., 2023, DeepSeek-AI, 2025, Zhou et al., 2025, Yu et al., 2025]. В то же время исследования оценки юмора показывают, что смешность является многомерной, контекстно-зависимой и лишь несовершенно захватывается модель оценщиков [Zhang et al., 2024, Sakabe et al., 2025, Winters and Van der Stockt, 2025, Hashemi et al., 2024].

Данная работа находится на пересечении этих результатов. Она утверждает, что юмор выигрывает от рассуждения, но отвергает необходимость задавать фиксированный творческий сценарий. Она также развивает без внешнего верификатора обучение с подкреплением, но утверждает, что творческие задачи требуют с несколькими эталонными ответами формулировка, поскольку несколько ответы могут быть одновременно хорошими, а наблюдаемый набор эталонных ответов неполон. Поэтому исследовательский вопрос состоит в том, можно ли адаптировать без внешнего верификатора рассуждение целевая функция к частично наблюдаемый с несколькими эталонными ответами

юмор генерация так, чтобы поощрять творческий поиск без схлопывания в запомненный обучение выборки.

Глава 2

Методология

2.1 Границы исследования и статус реализации

Методология данной предварительной дипломной работы имеет два уровня. Первый уровень — реализованный процесс подготовки данных и оценки, уже присутствующий в репозитории. Второй уровень — предлагаемая обучающая цель, а именно с несколькими эталонными ответами без внешнего верификатора (MRVF) обучение с подкреплением, которая математически сформулирована, но еще не полностью реализована в коде. Это различие важно для границ настоящей работы. Репозиторий уже содержит процессы для построения корпуса, генерации векторных представлений, извлечения ключевых слов, поиска эталонных ответов, генерации кандидатов и автоматической попарной оценки. Однако сам этап обучения остается будущей работой. Поэтому данная глава различает то, что уже операционально в кодовой базе, и то, что предлагается как следующий этап проекта.

На высоком уровне реализованная система строит слабо размеченный набор данных с несколькими эталонными ответами из сырых наборов данных шуток. Сначала шутки объединяются в единый корпус, затем кодируются моделью плотного векторного поиска и сводятся к небольшому набору ключевых слов. Эти ключевые слова расширяются в группы запросов на основе n -грамм, которые затем используются для поиска семантически близких шуток в индексе векторных представлений. Полученные эталонные ответы имеют два непосредственных применения. Во-первых, они дают данные для генерации базовой моделью и автоматической оценки. Во-вторых, они задают обучающую и валидационную выборки, необходимые для предлагаемой целевой функции обучения MRVF.

2.2 Построение набора данных

Набор данных объединяется из двух публичных коллекций шуток: наборов данных Short Jokes и rJokes. В репозитории этот этап реализован в `src/pipelines/jokes.py`. Конвейер скачивает сырые файлы из зафиксированных версий источников, преобразует их в формат Parquet и сохраняет метаданные источника для каждой шутки. Для каждой строки сохраняются поля `id`, `text`, `source_name`, `source_filename` и `source_id`. После отдельной предобработки двух корпусов они конкатенируются и получают новый глобальный целочисленный идентификатор. Итоговый набор данных содержит примерно 664 k объектов.

Такой дизайн намеренно является консервативным. Процесс не выполняет агрессивную нормализацию текста, кроме удаления пустых записей и стандартизации формата хранения. Для юмора такая осторожность желательна, поскольку кажущиеся мелкими текстовыми детали, такие как пунктуация, формулировка или переносы строк, могут влиять на комический эффект. Сохранение метаданных источника также полезно для последующего анализа доменных различий, поиска дубликатов и возможных смещений, связанных с источником.

2.3 Плотный индекс векторных представлений

Следующий этап кодирует каждую шутку в плотном семантическом векторном пространстве. Он реализован в `src/pipelines/embeddings.py`. Текущая конфигурация по умолчанию использует модель Qwen3-Embedding-8B с размерностью векторного представления 1024, уменьшенной с исходных 4096 с помощью Matryoshka Representation Learning. Векторные представления вычисляются асинхронно пакетами, записываются в фрагменты Parquet и хранятся как пары (i, e_i) , где i — глобальный идентификатор шутки, а $e_i \in \mathbb{R}^{1024}$ — соответствующее векторное представление.

Роль этого этапа двоякая. Во-первых, векторные представления шуток затем используются для оценки ключевых слов-кандидатов по семантической релевантности. Во-вторых, они задают векторный индекс, используемый для поиска наборов шуток с несколькими эталонными ответами. Иными словами,

пространство векторных представлений является общим представлением, от которого зависят и извлечение ключевых слов, и поиск эталонных ответов.

2.4 Извлечение ключевых слов как поиск слабых запросов

Предполагается, что каждая шутка имеет набор слабых запросов, на которые она правдоподобно отвечает. Например, “write a joke about a chicken” рассматривается как допустимый запрос для шутки “why did the chicken cross the road”. Этап извлечения ключевых слов направлен на построение слабых запросов на основе ключевых слов шуток. Он реализован в `src/pipelines/keywords.py`.

Для каждой шутки y_i процесс сначала генерирует пул n -грамм-кандидатов с помощью анализатора `CountVectorizer`. В конфигурации по умолчанию этот анализатор использует только униграммы, удаляет английские стоп-слова и оставляет не более 64 терминов-кандидатов на шутку. Пусть полученный набор кандидатов равен

$$C(y_i) = \{c_{i1}, \dots, c_{im}\}.$$

Затем каждый термин-кандидат кодируется той же моделью векторных представлений, что и шутки. Если $e(y_i)$ обозначает векторное представление шутки, а $e(c)$ обозначает векторное представление ключевого слова-кандидата, то оценка релевантности вычисляется через косинусное сходство:

$$s(c, y_i) = \frac{e(c)^\top e(y_i)}{\|e(c)\| \|e(y_i)\|}.$$

Выбор только наиболее похожих кандидатов часто давал бы избыточные ключевые слова. Для уменьшения избыточности процесс применяет `maximal marginal relevance (MMR)`, выбирая несколько ключевых слов с балансом между релевантностью и разнообразием. В конфигурации по умолчанию коэффициент разнообразия равен 0.7, а число сохраняемых ключевых слов равно 3. Поэтому выход этого этапа — компактное представление в

виде ключевых слов

$$K(y_i) = \{k_{i1}, k_{i2}, k_{i3}\},$$

которое может содержать меньше трех элементов, если пул кандидатов мал.

Методологически эти ключевые слова не следует интерпретировать как полноценные скрытые запросы. Это более слабые объекты: n-граммы, которые захватывают тематические или лексические якоря шуток. Это прагматическое приближение. Оно не решает восстановление запроса по шутке полностью, но дает удобное представление, из которого можно построить несколько вариантов запроса.

2.5 Построение эталонных ответов через группировку ключевых слов и поиск

2.5.1 Группировка ключевых слов и формирование запросов

Этап построения эталонных ответов реализован в `src/pipelines/references.py`. Для каждого набора ключевых слов $K(y_i)$ процесс перечисляет все уникальные комбинации ключевых слов, размер которых лежит между `min_keywords` и `max_keywords`. В конфигурации по умолчанию эти значения равны 1 и 2, поэтому процесс строит все однословные и двухсловные группы. Если

$$K(y_i) = \{k_1, k_2, k_3\},$$

то сгенерированные группы равны

$$\{k_1\}, \{k_2\}, \{k_3\}, \{k_1, k_2\}, \{k_1, k_3\}, \{k_2, k_3\}.$$

Каждая группа преобразуется в запрос на естественном языке с помощью шаблона

Write a joke using the following keyword(s): {keywords}

Этот запрос служит двум целям: это текстовая форма, которая затем подается генеративным моделям, и это же объект, который кодируется как запрос

для поиска эталонных ответов.

Такой дизайн создает контролируемый, но разнообразный спектр запросов. Запросы с одним ключевым словом являются широкими, тематическими и часто допускают много возможных шуток. Запросы с двумя ключевыми словами более ограничены и обычно извлекают более точные кандидаты. Текущая реализация останавливается на двух ключевых слов, потому что более крупные группы быстро становятся слишком специфичными, уменьшая число правдоподобных различных эталонных ответов на запрос.

2.5.2 Поиск эталонных ответов с помощью индекса векторных представлений

После формирования строк запросов процесс кодирует их как запросы и извлекает близкие шутки из индекса FAISS, построенного по полной коллекции векторных представлений шуток. Индекс поиска является инвертированным файловым индексом (IVF). Перед индексацией и поиском векторы нормализуются, так что скалярное произведение соответствует косинусному сходству. В конфигурации по умолчанию основные параметры таковы:

- модель векторных представлений: Qwen3-Embedding-8B;
- размерность: 1024;
- `faiss_nlist`: 4096;
- `faiss_nprobe`: 32;
- `top_k`: 5;
- `oversample`: 10;
- `min_similarity`: 0.5.

Для сформированного запроса x пусть $q(x)$ — его нормализованное векторное представление, а e_j — нормализованное векторное представление шутки y_j . Тогда оценка поиска равна

$$s(x, y_j) = q(x)^\top e_j.$$

Процедура поиска извлекает больше, чем k кандидатов, фильтрует те, чье сходство ниже порога, и оставляет k лучших оставшихся объектов. Соответствующие тексты затем используются как наблюдаемый набор эталонных ответов:

$$Y^*(x) = \{y_1, \dots, y_n\}, \quad n \leq 5.$$

На практике поиск эталонных ответов дает около 4 эталонных ответов на группу ключевых слов после удаления дублей и разбиения выборки. Целевой объем выборки для обучения равен 10 – 20k эталонных ответов, поэтому достаточно сгенерировать 3 – 5k групп ключевых слов.

Этот этап реализует центральную идею работы: запрос связывается с несколькими шутками-кандидатами в качестве эталонных ответов, а не с одним истинным ответом. Важно, что найденный набор не является исчерпывающе размеченным множеством всех хороших шуток для x . Это приближение, полученное поиском такого множества. Это существенно и эмпирически, и теоретически. Эмпирически найденные эталонные ответы могут содержать шум или пропускать валидные альтернативы. Теоретически неполнота $Y^*(x)$ становится ключевой частью дальнейшего анализа MRVF, а не второстепенным неудобством.

2.5.3 Удаление дублей и разбиение выборки

После поиска итоговый набор данных содержит строки вида

$$(x_i, Y^*(x_i), S(x_i)),$$

где x_i — запрос на основе ключевых слов, $Y^*(x_i)$ — найденный набор эталонных ответов, а $S(x_i)$ — вектор оценок поиска. Разные исходные шутки могут генерировать одинаковые группы ключевых слов, поэтому репозиторий дедуплицирует набор данных по полю **keywords**, сохраняет первое вхождение и заново назначает идентификаторы строк.

Дедуплицированный набор данных затем разбивается на обучающую, валидационную и тестовую части. Текущая реализация использует двухэтапное случайное разбиение с `test_fraction=0.1`, `validation_fraction=0.1` и фиксированным зерном случайности. Отдельные фрагменты Parquet записываются для полного набора данных и для каждой части выборки.

2.6 Генерация кандидатов

Этот этап поддерживает генерацию кандидатов на основе набора эталонных ответов и реализован в `src/pipelines/candidates.py`. Для выбранной модели и части набора данных процесс проходит по запросам и вызывает интерфейс генерации ответов, совместимый с OpenAI. Используется тот же шаблон запроса, что и при построении эталонных ответов:

```
Write a joke using the following keyword(s): {keywords}
```

В конфигурации по умолчанию генерация кандидатов использует температуру 1.0 и допускает до 128 токенов завершения. Важно сохранять шутки короткими, чтобы упростить оценку как с помощью языковой модели-оценщика, так и в условиях человеческой разметки.

Схема выхода содержит четыре поля: идентификатор строки, группа ключевых слов, название модели и сгенерированный текст. Файлы с кандидатами сохраняются в структуре каталогов по модели и части выборки. Такая организация важна для последующей оценки, поскольку позволяет выравнивать ответы нескольких моделей по одному базовому идентификатору и запросу.

Методологически этот этап генерации кандидатов следует понимать как слой контрольного набора. Он создает артефакты, необходимые для сравнения базовых моделей: предобученная базовая модель, версий в режимах `instruct` и `thinking`, а также будущих обученных контрольных точек.

2.7 Оценка кандидатов

2.7.1 Попарное сравнение

Автоматическая оценка реализована в `src/pipelines/evaluation.py`. Процесс сначала конкатенирует наборы данных с кандидатами от нескольких моделей. Затем он группирует кандидаты по идентификатору эталонного набора и название модели и строит попарные сравнения для всех пар моделей, которые произвели ответы для одного запроса. Порядок левого и правого ответа рандомизируется с фиксированным зерном случайности, чтобы уменьшить смещение из-за позиции ответа.

Оценочный запрос намеренно прост: он содержит общий запрос на шутку, а также левый и правый кандидаты. Системная инструкция просит разметчик действовать как оценщик качества юмора, выбрать, какая шутка смешнее для широкой аудитории, избегать ничьих и возвращать строгий JSON. В конфигурации по умолчанию модель-оценщик работает при температуре 0.

Этот слой оценки захватывает только сравнительную смешность при общем запросе. Он пока не реализует многомерную оценку, обсуждаемую в обзоре литературы: отдельные суждения по рубрикам для соответствия запросу, новизны, похожесть на человеческий ответ и смешность; также он пока не включает человеческую разметку. Эти этапы остаются будущей работой, поскольку зависят от завершенной процедуры обучения.

2.7.2 Агрегация таблицы результатов

Попарные суждения агрегируются в рейтинг с числом побед, числом поражений, числом сравнений, долей побед и оценками Bradley-Terry. Пусть $\mathcal{M}$ обозначает множество оцениваемых моделей. Для каждой оцениваемой пары (m_a, m_b) оценщик записывает бинарную победу для одной стороны. Затем процесс оценивает параметры Bradley-Terry и сообщает их логарифмы как значения `bt_score`.

Такая агрегация разумна для юмора, потому что относительные суждения часто стабильнее абсолютных оценок. Судье обычно легче решить, какая из двух шуток смешнее, чем дать хорошо калиброванную числовую оценку. В то же время эта процедура наследует ограничения модель-оценщик. Поскольку суждения о юморе языковых моделей могут расходиться с человеческими суждениями, текущий рейтинг следует интерпретировать как быстрый и дешевую приближенную меру для промежуточной валидации, а не как окончательную меру качества юмора.

2.8 Теоретическая рамка обучения MRVF

Реализованный выше процесс строит наборы эталонных ответов, обусловленные запросом; данный раздел определяет, как такие множества входят в целевую функцию обучения с подкреплением, ориентированного на рассуж-

дение. Центральное утверждение состоит в том, что генерация юмора плохо согласуется с предположениями об одном эталонном ответе, используемыми в стандартном основанном на верификаторе или без внешнего верификатора обучении с подкреплением для рассуждения. Слабый запрос может допускать множество приемлемых шуток, и репозиторий уже делает это конкретным, извлекая несколько эталонных ответов для одного запроса на основе ключевых слов. Ниже формализуется оптимизация по такой разметке в виде множества допустимых ответов.

2.8.1 Обозначения

Пусть $\mathcal{X}$, $\mathcal{Z}$ и $\mathcal{Y}$ обозначают пространства запросов, траекторий рассуждения и итоговых ответов соответственно. Для запроса $x \in \mathcal{X}$:

- $z \in \mathcal{Z}$ обозначает траекторию рассуждения,
- $y \in \mathcal{Y}$ обозначает итоговую сгенерированную шутку,
- $y^* \in \mathcal{Y}$ обозначает один эталонный ответ,
- $Y^*(x) \subset \mathcal{Y}$ обозначает *наблюдаемый* набор допустимых эталонных ответов, связанный с запросом x ,
- π_θ обозначает стратегия, то есть условную вероятностную модель над текстом, параметризованную θ .

Более явно,

$$\pi_\theta(z, y \mid x) = P_\theta(Z = z, Y = y \mid X = x).$$

Стратегия факторизуется как

$$\pi_\theta(z, y \mid x) = \pi_\theta(z \mid x) \pi_\theta(y \mid x, z).$$

Когда модель явно выводит токены рассуждения, z можно отождествить с текстом внутри специального сегмента `<think>`, а y — с итоговым ответом, который следует после него. Когда рассуждение явно не экспонируется в выходной поток, ту же факторизацию можно понимать как концептуальное

разложение на выбранную промежуточную траекторию z и итоговый ответ, обусловленный этой траекторией. Преимущество такой нотации в том, что она разделяет две роли модели: выбор пути рассуждения и распределение вероятностной массы по ответам при данном пути.

Для краткости целевая функция оптимизации ниже записывается через наблюдаемый набор $Y^*(x)$. При обсуждении неполноты будет полезно представлять более крупное скрытое множество $Y_{\text{true}}(x) \supseteq Y^*(x)$, содержащее все действительно приемлемые шутки для запроса x , хотя это полное множество никогда не наблюдается на практике.

2.8.2 RL с верифицируемыми наградами

Обучение с подкреплением оптимизирует стратегию, максимизируя ожидаемую награду. При заданной целевая функция $J(\theta)$ обучение идет через stochastic градиент восхождение или, эквивалентно, через градиентный спуск по отрицательной целевой функции. Поэтому критическое проектное решение — функция награды: она определяет, какие сгенерированные ответы получают положительный сигнал.

В проверяемых областях награда может быть назначена внешней процедурой, не зависящей от субъективного человеческого суждения. Для кода сгенерированную программу можно выполнить на набор тестов; для математики итоговый ответ можно распарсить и сравнить с каноническим решением. В таких случаях событие “ответ является правильным” имеет смысл, внешний по отношению к самой языковой модели. Современные модели рассуждения сильно опираются на эту структуру при обучении [DeepSeek-AI, 2025, Lightman et al., 2023].

Сначала рассмотрим стандартную постановку с одним эталонным ответом: для каждого запроса x существует ровно одна проверяемая цель y^* . Определим награду

$$R(y, y^*) = \mathbb{I}\{y = y^*\},$$

где $\mathbb{I}\{\cdot\}$ — индикаторная функция. Тогда целевая функция равна

$$J_{\text{RLVR}}(\theta \mid x, y^*) = \mathbb{E}_{z \sim \pi_\theta(z|x)} \mathbb{E}_{y \sim \pi_\theta(y|x,z)} [R(y, y^*)].$$

Эквивалентно, в развернутой форме суммы,

$$J_{\text{RLVR}}(\theta \mid x, y^*) = \sum_{z \in \mathcal{Z}} \sum_{y \in \mathcal{Y}} \pi_{\theta}(z \mid x) \pi_{\theta}(y \mid x, z) R(y, y^*).$$

Для оптимизации этого математического ожидания используется тождество функции оценки, также известная как тождество REINFORCE:

$$\nabla_{\theta} \mathbb{E}_{u \sim p_{\theta}(u)}[f(u)] = \mathbb{E}_{u \sim p_{\theta}(u)}[f(u) \nabla_{\theta} \log p_{\theta}(u)].$$

Это тождество следует напрямую из соотношения $\nabla_{\theta} p_{\theta}(u) = p_{\theta}(u) \nabla_{\theta} \log p_{\theta}(u)$. Применяя его к совместному распределению (z, y) , получаем

$$\begin{aligned} \nabla_{\theta} J_{\text{RLVR}} &= \nabla_{\theta} \sum_{z, y} \pi_{\theta}(z, y \mid x) R(y, y^*) \\ &= \sum_{z, y} R(y, y^*) \nabla_{\theta} \pi_{\theta}(z, y \mid x) \\ &= \sum_{z, y} \pi_{\theta}(z, y \mid x) R(y, y^*) \nabla_{\theta} \log \pi_{\theta}(z, y \mid x) \\ &= \mathbb{E}_{z, y} \left[R(y, y^*) \nabla_{\theta} \log \pi_{\theta}(z, y \mid x) \right]. \end{aligned}$$

Используя факторизацию совместной стратегии,

$$\log \pi_{\theta}(z, y \mid x) = \log \pi_{\theta}(z \mid x) + \log \pi_{\theta}(y \mid x, z),$$

следовательно,

$$\nabla_{\theta} \log \pi_{\theta}(z, y \mid x) = \nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log \pi_{\theta}(y \mid x, z).$$

Поэтому

$$\nabla_{\theta} J_{\text{RLVR}} = \mathbb{E}_{z, y} \left[R(y, y^*) (\nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log \pi_{\theta}(y \mid x, z)) \right].$$

Это выражение имеет простую интерпретацию. Выбранная пара (z, y) получает награду только тогда, когда верификатор объявляет y правильным. Когда это происходит, обновление увеличивает и вероятность траектория рассуждения, приведшего к ответу, и вероятность самого ответа при этом

траектория.

2.8.3 VeriFree

Юмор не предоставляет такого внешнего верификатора, и в более общем виде многие задачи открытой генерации не имеют детерминированного теста правильности. VeriFree закрывает этот разрыв, замечая, что в случае одного эталонного ответа ожидаемый точное совпадение награда можно переписать как условная вероятность [Zhou et al., 2025].

Зафиксируем x и z . Тогда

$$\begin{aligned}\mathbb{E}_{y \sim \pi_\theta(\cdot | x, z)} [\mathbb{I}\{y = y^*\}] &= \sum_{y \in \mathcal{Y}} \pi_\theta(y | x, z) \mathbb{I}\{y = y^*\} \\ &= \pi_\theta(y^* | x, z).\end{aligned}$$

Подставляя это тождество обратно в RLVR целевая функция, получаем

$$J_{\text{RLVR}}(\theta | x, y^*) = \mathbb{E}_{z \sim \pi_\theta(z | x)} [\pi_\theta(y^* | x, z)].$$

Это мотивирует награду без внешнего верификатора

$$r_\theta(x, z, y^*) := \pi_\theta(y^* | x, z),$$

и соответствующую целевая функция

$$J_{\text{VF}}(\theta | x, y^*) := \mathbb{E}_{z \sim \pi_\theta(z | x)} [r_\theta(x, z, y^*)].$$

При с одним эталонным ответом предположение

$$J_{\text{VF}}(\theta | x, y^*) \equiv J_{\text{RLVR}}(\theta | x, y^*)$$

как скалярные целевые функции. Значимость такой переформулировки одновременно практическая и концептуальная: для вычисления награды не требуется явное декодирование y , а $\pi_\theta(y^* | x, z)$ можно вычислить с помощью принудительного учительского декодирования на последовательность эталонного ответа, что удобно параллелизуется.

Чтобы получить стратегия градиент, запишем

$$J_{\text{VF}}(\theta \mid x, y^*) = \sum_{z \in \mathcal{Z}} \pi_{\theta}(z \mid x) r_{\theta}(x, z, y^*).$$

Дифференцируя по правилу произведения,

$$\begin{aligned} \nabla_{\theta} J_{\text{VF}} &= \sum_z \left[\nabla_{\theta} \pi_{\theta}(z \mid x) r_{\theta}(x, z, y^*) + \pi_{\theta}(z \mid x) \nabla_{\theta} r_{\theta}(x, z, y^*) \right] \\ &= \sum_z \pi_{\theta}(z \mid x) \left[r_{\theta}(x, z, y^*) \nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} r_{\theta}(x, z, y^*) \right]. \end{aligned}$$

Так как $r_{\theta}(x, z, y^*) = \pi_{\theta}(y^* \mid x, z)$,

$$\nabla_{\theta} r_{\theta}(x, z, y^*) = \pi_{\theta}(y^* \mid x, z) \nabla_{\theta} \log \pi_{\theta}(y^* \mid x, z) = r_{\theta}(x, z, y^*) \nabla_{\theta} \log \pi_{\theta}(y^* \mid x, z).$$

Следовательно,

$$\nabla_{\theta} J_{\text{VF}} = \mathbb{E}_{z \sim \pi_{\theta}(z \mid x)} \left[r_{\theta}(x, z, y^*) (\nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log \pi_{\theta}(y^* \mid x, z)) \right].$$

Эта декомпозиция информативна. Первый член,

$$\mathbb{E}_z [r_{\theta}(x, z, y^*) \nabla_{\theta} \log \pi_{\theta}(z \mid x)],$$

является слагаемое функции оценки по выбранный траектории рассуждения. Он повышает вероятность траектории, при которых сама модель назначает более высокую условный масса эталонный ответ. Второй член,

$$\mathbb{E}_z [\nabla_{\theta} r_{\theta}(x, z, y^*)],$$

является прямой teacher-forced градиент на последовательность эталонного ответа. По сравнению с RLVR, VeriFree убирает шум от выборки итогового ответа y : оценщик все еще выборки z , но не ответ, используемый для оценка награды. Именно поэтому VeriFree привлекателен в доменах, где верификатор недоступен, но один эталонный ответ существует.

2.8.4 С несколькими эталонными ответами VeriFree

Один-эталонный ответ предположение не подходит для юмора. Запрос вроде “напиши шутка используя ключевые слова cat, dog” не имеет одного правильного продолжение; он имеет много правдоподобных продолжений, различающихся завязка, механизм панчлайна, тон и уровнем абсурдности. Фактически реализованный репозиторий явно предназначен для построения нескольких эталонных ответов для одного слабого запроса. Поэтому награда должна быть определен на множестве допустимых ответах, а не на одном ответе.

Пусть $Y^*(x) \subset \mathcal{Y}$ обозначает наблюдаемый набор допустимый эталонные ответы для запроса x . Определим проверяемую награду с несколькими эталонными ответами

$$R(y, Y^*) = \mathbb{I}\{y \in Y^*\}.$$

Соответствующая set-valued точное совпадение целевая функция равна

$$J_{\text{MR}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x, z)} [\mathbb{I}\{y \in Y^*\}].$$

Для фиксированных x и z ,

$$\begin{aligned} \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x, z)} [\mathbb{I}\{y \in Y^*\}] &= \sum_{y \in \mathcal{Y}} \pi_\theta(y \mid x, z) \mathbb{I}\{y \in Y^*\} \\ &= \sum_{y \in Y^*} \pi_\theta(y \mid x, z). \end{aligned}$$

Это мотивирует с несколькими эталонными ответами награду без внешнего верификатора

$$r_\theta(x, z, Y^*) := \sum_{y \in Y^*} \pi_\theta(y \mid x, z).$$

Величина $r_\theta(x, z, Y^*)$ лежит в $[0, 1]$: это total вероятностная масса, которую стратегия назначает, при траектория рассуждения z , наблюдаемый набор эталонных ответов. Эквивалентно, это вероятность события, что сгенерированный ответ попадает внутрь этого множества. Итоговая целевая функция равна

$$J_{\text{MRVF}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z \mid x)} [r_\theta(x, z, Y^*)].$$

На уровне скалярной целевой функции MRVF играет для set-valued на-

града ту же роль, что VeriFree играет для single точное совпадение награда. Однако градиент structure не является просто с одним эталонным ответом формулой, где y^* заменено символом множества. Со стороны ответа производная меняется существенно.

Действительно,

$$J_{\text{MRVF}}(\theta \mid x, Y^*) = \sum_z \pi_\theta(z \mid x) r_\theta(x, z, Y^*),$$

поэтому

$$\begin{aligned} \nabla_\theta J_{\text{MRVF}} &= \sum_z \left[\nabla_\theta \pi_\theta(z \mid x) r_\theta(x, z, Y^*) + \pi_\theta(z \mid x) \nabla_\theta r_\theta(x, z, Y^*) \right] \\ &= \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \left[r_\theta(x, z, Y^*) \nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta r_\theta(x, z, Y^*) \right]. \end{aligned}$$

Теперь раскроем по эталонным ответам производная:

$$\begin{aligned} \nabla_\theta r_\theta(x, z, Y^*) &= \nabla_\theta \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \\ &= \sum_{y \in Y^*} \nabla_\theta \pi_\theta(y \mid x, z) \\ &= \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \nabla_\theta \log \pi_\theta(y \mid x, z). \end{aligned}$$

Поэтому

$$\nabla_\theta J_{\text{MRVF}} = \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \left[r_\theta(x, z, Y^*) \nabla_\theta \log \pi_\theta(z \mid x) + \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \nabla_\theta \log \pi_\theta(y \mid x, z) \right].$$

Иногда полезно переписать второй член в нормализованной форме. Если $r_\theta(x, z, Y^*) > 0$, определим

$$w_\theta(y \mid x, z, Y^*) := \frac{\pi_\theta(y \mid x, z)}{r_\theta(x, z, Y^*)} \mathbb{I}\{y \in Y^*\}.$$

Тогда $w_\theta(\cdot \mid x, z, Y^*)$ является вероятностное распределение over наблюдаемый

набор эталонных ответов, и

$$\nabla_{\theta} r_{\theta}(x, z, Y^*) = r_{\theta}(x, z, Y^*) \sum_{y \in Y^*} w_{\theta}(y \mid x, z, Y^*) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z).$$

Эквивалентно,

$$\nabla_{\theta} \log r_{\theta}(x, z, Y^*) = \sum_{y \in Y^*} w_{\theta}(y \mid x, z, Y^*) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z),$$

и, следовательно,

$$\nabla_{\theta} J_{\text{MRVF}} = \mathbb{E}_{z \sim \pi_{\theta}(z|x)} \left[r_{\theta}(x, z, Y^*) (\nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log r_{\theta}(x, z, Y^*)) \right].$$

Эта форма проясняет отличие от с одним эталонным ответом VeriFree. В VeriFree прямой term — это градиент $\log \pi_{\theta}(y^* \mid x, z)$ для одного эталонного ответа. В MRVF прямой term — это градиент $\log \sum_{y \in Y^*} \pi_{\theta}(y \mid x, z)$. Поэтому обновление не отдаёт априорного предпочтения одному эталонный ответ; он увеличивает полная вероятностная масса на наблюдаемый набор, причем каждый эталонный ответ взвешивается текущей вероятностной массе стратегии, назначенной ему. Если один эталонный ответ уже доминирует, градиент концентрируется на нем. Если масса распределена по нескольким эталонные ответы, градиент усиливает их совместно. Поэтому MRVF не является просто нотационным вариантом VeriFree: оптимизация геометрия меняется с одной целевая строка на логарифм суммы по многим целевым строкам.

2.8.5 Неполные наблюдаемые эталонные ответы

Наблюдаемый set $Y^*(x)$ все еще не является полным множеством хороших шуток. Возникают две разные формы неполнота.

Структурная неполнота. Даже если репозиторий извлекает несколько эталонные ответы для запрос, может существовать много дополнительных шуток в скрытое множество $Y_{\text{true}}(x)$, которые также валидны, но не наблюдаются. Это неизбежно в творческий области. Конечный набор данных не может перечислить все допустимые панчлайны для слабый запрос.

Вычислительная неполнота. Даже внутри наблюдаемый набор $Y^*(x)$ вычисление

$$r_\theta(x, z, Y^*) = \sum_{y \in Y^*} \pi_\theta(y \mid x, z)$$

может стать дорогим, когда $|Y^*|$ велико. Поэтому сумму можно оценивать на выбранный subset $Y \subseteq Y^*$.

Предположим, что Y выбранный uniformly without замена из Y^* , и каждый эталонный ответ $y \in Y^*$ имеет вероятность включения

$$\rho = P(y \in Y) = \frac{|Y|}{|Y^*|}.$$

Определим subset награда

$$r_\theta(x, z, Y) := \sum_{y \in Y} \pi_\theta(y \mid x, z) = \sum_{y \in Y^*} \mathbb{I}\{y \in Y\} \pi_\theta(y \mid x, z).$$

Беря математическое ожидание по случайному подмножеству,

$$\begin{aligned} \mathbb{E}_Y [r_\theta(x, z, Y)] &= \sum_{y \in Y^*} \mathbb{E}_Y [\mathbb{I}\{y \in Y\}] \pi_\theta(y \mid x, z) \\ &= \sum_{y \in Y^*} P(y \in Y) \pi_\theta(y \mid x, z) \\ &= \rho \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \\ &= \rho r_\theta(x, z, Y^*). \end{aligned}$$

Следовательно, subset награда пропорционален в математическое ожидание полный по наблюдаемому набору награда.

Здесь становится релевантным использование базового уровня в GRPO. Предположим, что для фиксированного запрос x все выбранные награды в группе умножаются на один и тот же положительный множитель ρ_x , например потому что используется только доля наблюдаемые эталонные ответы. Если стандартизовать награды внутри группы,

$$\tilde{A}_i = \frac{\hat{r}_i - \mu_x}{\sigma_x + \varepsilon}, \quad \mu_x = \frac{1}{G} \sum_{j=1}^G \hat{r}_j,$$

то умножение каждого $\hat{r}_i$ на $\rho_x > 0$ умножает и числитель, и знаменатель на один и тот же множитель, оставляя $\tilde{A}_i$ неизменным. В этом смысле GRPO группа standardization масштаб-invariant. Она смягчает сдвиги масштаба награды в слагаемое функции оценки, хотя не восстанавливает информацию, потерянную из-за пропущенные эталонные ответы, и сама по себе не решает смещение прямого градиента по эталонным ответам.

Другая фундаментальная проблема — проблема ложных отрицательных примеров. Оптимизация

$$r_\theta(x, z, Y^*) = \sum_{y \in Y^*} \pi_\theta(y \mid x, z)$$

увеличивает вероятностная масса на наблюдаемые эталонные ответы. Поскольку условное распределение по всем строкам суммируется в единицу,

$$\sum_{y \in \mathcal{Y}} \pi_\theta(y \mid x, z) = 1,$$

увеличенная масса должна прийти из $\mathcal{Y} \setminus Y^*$. К сожалению, это дополнение содержит не только плохие шутки, но и ненаблюдаемые хорошие шутки из $Y_{\text{true}}(x) \setminus Y^*(x)$. Иными словами, обучение signal может непреднамеренно подавлять валидные, но ненаблюдаемые удачные юмористические продолжения. Этот риск не является дефектом градиент вывод; он является структурным следствием обучение from неполная разметка в область с естественно большим числом допустимых решений.

Чтобы уменьшить этот риск, предлагаемая оптимизация включает штраф Кульбака–Лейблера относительно опорной стратегии π_{ref} :

$$\max_{\theta} \mathbb{E}_{x,z} [r_\theta(x, z, Y^*(x))] - \beta \mathbb{E}_x [\text{KL}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))].$$

Эквивалентно, можно рассматривать задачу в форме задачи с ограничением,

$$\max_{\theta} \mathbb{E}_{x,z} [r_\theta(x, z, Y^*(x))] \quad \text{s.t.} \quad \mathbb{E}_x [\text{KL}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))] \leq \varepsilon.$$

В данном контексте KL-слагаемое не является просто обычной регуляризацией. Его методологическая роль — препятствовать чрезмерно агрессивному

перераспределению вероятностной массы от опорной модели, когда наблюдаемые наборы эталонных ответов заведомо неполны.

2.8.6 Предлагаемый цикл обучения

Предполагаемый этап обучения выборки несколько траектории рассуждения для одного запроса и использует их награды для обновления стратегия рассуждения. Для запроса x выборка группа

$$z_1, \dots, z_G \sim \pi_\theta(z \mid x).$$

Для каждого z_i вычисляется награда по нескольким эталонным ответам

$$\hat{r}_i := r_\theta(x, z_i, Y^*(x)) = \sum_{y \in Y^*(x)} \pi_\theta(y \mid x, z_i).$$

На практике каждая $\pi_\theta(y \mid x, z_i)$ получается с помощью принудительного учительского декодирования на последовательность эталонного ответа y . Поскольку эти вероятности последовательностей очень малы, реализация должна аккумулировать логарифмы вероятностей токенов и, где необходимо, использовать стабилизацию вроде $\log \sum \exp$ перед обратным переходом к вероятностям или нормализованным весам.

Градиент естественно раскладывается на три части:

$$\nabla_\theta J_{\text{MRVF}} = g_z + g_r + g_{\text{KL}}.$$

Рассуждение term. Функция оценки часть по z оценивается по выбранный траектории и поэтому выигрывает от снижение дисперсии. Чистый выбор — с исключением текущего элемента базовая модель

$$b_i = \frac{1}{G-1} \sum_{j \neq i} \hat{r}_j, \quad A_i = \hat{r}_i - b_i.$$

Соответствующий оценщик равен

$$g_z = \frac{1}{G} \sum_{i=1}^G A_i \nabla_\theta \log \pi_\theta(z_i \mid x).$$

Это та же логика, которая мотивирует RLOO оценители: каждая траектория сравнивается с другими траектории, выбранный для того же запроса. При желании можно дополнительно нормализовать A_i внутри группы, чтобы уменьшить запрос-специфическую изменчивость масштаба награды, в духе GRPO. Такая нормализация повышает устойчивость, но уже не является строго несмещенной, поскольку масштаб оценки сама зависит от выбранной группы.

Эталонный ответ term. Для прямой ответ-side вклад корректный градиент по наблюдаемому набору равен

$$g_r = \frac{1}{G} \sum_{i=1}^G \nabla_{\theta} r_{\theta}(x, z_i, Y^*(x)) = \frac{1}{G} \sum_{i=1}^G \sum_{y \in Y^*(x)} \pi_{\theta}(y \mid x, z_i) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z_i).$$

Эквивалентно, используя нормализованные веса внутри набора

$$w_{i,y} := \frac{\pi_{\theta}(y \mid x, z_i)}{\hat{r}_i} \mathbb{I}\{y \in Y^*(x)\},$$

можно записать

$$g_r = \frac{1}{G} \sum_{i=1}^G \hat{r}_i \sum_{y \in Y^*(x)} w_{i,y} \nabla_{\theta} \log \pi_{\theta}(y \mid x, z_i),$$

если $\hat{r}_i > 0$. Эта форма делает геометрия явной: каждая траектория получает total weight $\hat{r}_i$, а внутри этого траектория градиент распределяется по эталонным ответам согласно текущей вероятностной массе стратегии на каждом эталонный ответ.

KL-слагаемое. Наконец,

$$g_{\text{KL}} = -\beta \nabla_{\theta} \text{KL}(\pi_{\theta}(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x)).$$

Полный обновление для мини-пакет усредняет эти запрос-уровень вклады по выбранный запросы.

Практическая важность реализованного процесса теперь ясна. Без наборы, обусловленные запросом с несколькими эталонными ответами $Y^*(x)$

нельзя вычислить ни $\hat{r}_i$, ни g_r . Поэтому процесс подготовки данных не является вспомогательным к теории; он предоставляет наблюдаемый structure, от которой зависит MRVF оптимизация.

2.8.7 Замечание о использовании базового уровня

Дисперсия снижение прямолинейно применима только к слагаемое функции оценки. Причина в том, что z действительно выбранный from $\pi_\theta(z | x)$. Если $b(x, z_{-i})$ — базовая модель, не зависящий от выбранный variable z_i , то

$$\begin{aligned}\mathbb{E}_{z_i \sim \pi_\theta(\cdot | x)} [b(x, z_{-i}) \nabla_\theta \log \pi_\theta(z_i | x)] &= b(x, z_{-i}) \sum_{z_i} \pi_\theta(z_i | x) \nabla_\theta \log \pi_\theta(z_i | x) \\ &= b(x, z_{-i}) \sum_{z_i} \nabla_\theta \pi_\theta(z_i | x) \\ &= b(x, z_{-i}) \nabla_\theta 1 \\ &= 0.\end{aligned}$$

Поэтому с исключением текущего элемента базовая модель несмещен для слагаемого рассуждения.

Эталонный ответ term устроен иначе. Это не математическое ожидание по выбранным ответам $y \sim \pi_\theta(\cdot | x, z)$, а прямой производная summed вероятность эталонного ответа. Следовательно, нет тождества вида

$$\mathbb{E}_y [\nabla_\theta \log \pi_\theta(y | x, z)] = 0$$

доступного для центрирования. Например, в случае одного эталонного ответа,

$$\mathbb{E}_z [(r_\theta(x, z, y^*) - b(x)) \nabla_\theta \log \pi_\theta(y^* | x, z)]$$

равно

$$\mathbb{E}_z [r_\theta(x, z, y^*) \nabla_\theta \log \pi_\theta(y^* | x, z)] - b(x) \mathbb{E}_z [\nabla_\theta \log \pi_\theta(y^* | x, z)],$$

и второй член в общем случае не равен нулю. Та же проблема сохраняется с несколькими эталонными ответами случай с $\nabla_\theta \log r_\theta(x, z, Y^*)$. Поэтому базовые модели, безвредные для слагаемое функции оценки, становятся смещенными, если наивно применить их к прямой градиент по эталонным

ответам.

Это различие важно для дизайна обучения алгоритм. Рассуждение term должен использовать variance-снижение механизмы, такую как с исключением текущего элемента базовые модели или аккуратно выбранную группу нормализация. Эталонный ответ term, напротив, следует вычислять максимально прямо из по наблюдаемому набору вероятностей, принимая, что это детерминированный, но потенциально смещенный приближенная мера для истинной награды, порождаемого $Y_{\text{true}}(x)$.

В совокупности эти наблюдения объясняют общую структуру предложения. MRVF наследует ключевое вычислительное преимущество обучения без внешнего верификатора: он избегает внешние верификаторы и явный награды выборка over итоговые ответы, но должен быть адаптирован к с несколькими эталонными ответами и неполному характеру юмора. Итоговая целевая функция математически вычислима, но ее ограничения являются частью вклада работы: неполнота, ложные отрицательные примеры и построение оценщика центральны для проблемы.

Глава 3

Результаты

3.1 Текущие результаты

Реализованный процесс уже поддерживает сбор корпуса, генерацию векторных представлений, извлечение ключевых слов, построение запросы, поиск эталонных ответов, генерацию кандидатов и автоматическая попарная оценка. Оставшийся отсутствующий этап — собственно обучения с подкреплением обновление цикл. Поэтому данный раздел является узким: он описывает завершенные артефакты данных и фиксирует конкретный экспериментальный постановка для первых MRVF обучающие запуски.

Результат на уровне корпуса — объединенная коллекция шуток примерно из 664k объектов. В финальной схема обучения этот полный корпус служит пул поиска: все шутки кодируются векторных представлений и индексируются, а эталонные ответы для слабые запросы извлекаются из этого полного пространства векторных представлений. Однако извлечение ключевых слов нужно применять только к достаточно большому подмножеству, чтобы достичь целевого объем выборки. В нашем случае 3 — 5k группы ключевых слов могут дать 10 — 20k эталонные ответы для обучение, валидация и тестирование.

Для предварительный этап этого достаточно. Важный результат состоит в том, что данные и экспериментальный постановка теперь финализированы: корпус существует, эталонный ответ коллекция реализована, методология зафиксирована, а оставшаяся работа ограничена цикл обучения MRVF.

3.2 Планируемая процедура обучения

Репозиторий уже реализует полный процесс подготовки данных на Python с использованием Hugging Face Наборы данных, Parquet артефакты, FAISS-based ближайших соседей поиск и OpenAI-compatible API вызовы для векторное представление генерация, генерацию кандидатов и автоматический оценка. Однако для собственно обучающие запуски стек программного обеспечения немного изменится.

Планируемая реализация будет использовать модифицированную версию `trl` для оптимизация и `vLLM` для генерация. `trl` предоставляет со стороны обучения логика для оптимизация стратегии, тогда как `vLLM` будет использоваться и для генерации ответы базовых моделей, и для обслуживания траектории генерации во время обучение с подкреплением. Поскольку MRVF зависит от выборка нескольких траектории рассуждения на запрос, пропускная способность генерации траекторий важен для эффективности.

Целевые модели относятся к семейству Qwen3 в диапазоне размеров 1.7B и 4B. Для каждого размера планируются следующие базовые модели:

1. **Qwen3 Instruct**, базовая модель в без режима рассуждения режим;
2. **Qwen3 Thinking**, базовая модель в thinking режим;
3. **MRVF-обученных Qwen3 Thinking**, предлагаемая модель;

Такая постановка сохраняет интерпретируемость сравнения. Он изолирует три отдельных вопроса: достаточно ли обычной инструкция настройка для генерации юмора; улучшает ли родной thinking режим генерацию юмора уже без дополнительного обучения; и может ли MRVF дополнительно улучшить thinking модель.

Все обучающие запуски планируются на 1–2 NVIDIA H200 GPUs в HSE HPC кластер. Этого достаточно для целевых размеров моделей и позволяет распараллеливать эксперименты между объекты.

Сравнение будет использовать процесс оценки, уже реализованный в репозитории. Ответы сопоставляются для одного запроса, порядок левого и правого вариантов рандомизируется, оценщик принуждается выбрать ровно одного победитель, а оценки моделей агрегируются через Bradley-Terry рей-

тинг. Этого достаточно для эксперимента, поскольку цель состоит в обнаружении того, дает ли MRVF измеримый сдвиг относительно instruct и базовые модели в режиме рассуждения. Более крупная многомерная оценка с оцениванием по рубрике и человеческую разметку планируется после завершения обучения.

3.3 Планируемые абляции

План абляций напрямую связан с теорией. Наиболее информативные абляции следующие:

- **Число наблюдаемых эталонных ответов на запрос $|Y^*|$.** Сравнить меньшие и большие наборы эталонных ответов, например через обрезку результатов поиска до 1, 3 или 5 объектов.
- **коэффициент KL β .** Проверить, насколько сильно модель должна быть удерживаться рядом с начальной контрольной точкой, чтобы компенсировать неполнота.
- **Размер группы G .** Варьировать число выбранных траектории рассуждения на один запрос, чтобы оценить variance / пропускная способность компромисс MRVF оценщик.
- **Размер модели.** Сравнить 1.7B и 4B, а также проверить, насколько MRVF устойчив при применении к меньшим моделям.

Список литературы

- Mostafa Amin and Manuel Burghardt. A survey on approaches to computational humor generation. In *Proceedings of the 4th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 29–41, 2020. URL <https://aclanthology.org/2020.latechclfl-1.4/>.
- Yahui Chen, Buxin Shi, and Meng Si. Prompt to GPT-3: Step-by-step thinking instructions for humor generation, 2023. URL <https://arxiv.org/abs/2306.13195>. arXiv:2306.13195.
- Tianyu Chu, Shuhua Zhang, Yu Chen, Ning Ding, and Tao Yu. SFT memorizes, RL generalizes: A comparative study of foundation model post-training, 2025. URL <https://openreview.net/forum?id=2P4rYTtGok>. OpenReview / arXiv preprint.
- Andrea Cocchieri, Lorenzo Ragazzi, Pietro Italiani, Giuseppe Tagliavini, and Gianluca Moro. “what do you call a dog that is incontrovertibly true? dogma”: Testing LLM generalization through humor. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22922–22937, 2025. URL <https://doi.org/10.18653/v1/2025.acl-long.1117>.
- Victor De Marez, Tim Winters, and Ayla Rigouts Terryn. THInC: A theory-driven framework for computational humor detection, 2024. URL <https://arxiv.org/abs/2409.01232>. arXiv:2409.01232.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL <https://doi.org/10.48550/arXiv.2501.12948>. arXiv:2501.12948.

- Leo Gao, John Schulman, Jacob Hilton, et al. Scaling laws for reward model overoptimization, 2023. URL <https://arxiv.org/abs/2210.10760>. arXiv:2210.10760.
- Hamid Hashemi, Jason Eisner, Corby Rosset, Benjamin Van Durme, and Chris Kedzie. LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://aclanthology.org/2024.acl-long.745/>.
- Justine T. Kao, Roger Levy, and Noah D. Goodman. A computational model of linguistic humor in puns. *Cognitive Science*, 40(5):1270–1285, 2016. URL <https://doi.org/10.1111/cogs.12269>.
- Sangho Kim and Lydia B. Chilton. AI humor generation: Cognitive, social and creative skills for effective humor, 2025. URL <https://arxiv.org/abs/2502.07981>. arXiv:2502.07981.
- Jens Lemmens and Victor De Marez. Computational humor modeling: A survey on the state of the art. *ACM Computing Surveys*, 58(7):177:1–177:37, 2026. URL <https://www.uantwerpen.be/en/staff/jens-lemmens/publications/>.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>. OpenAI / arXiv preprint.
- Thomas Loakman, William Thorne, and Chien-Sheng Lin. Who’s laughing now? an overview of computational humour generation and explanation, 2025a. URL <https://arxiv.org/abs/2509.21175>. arXiv:2509.21175.
- Thomas Loakman, William Thorne, and Chien-Sheng Lin. Comparing apples to oranges: A dataset & analysis of LLM humour understanding from traditional puns to topical jokes. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 9502–9518, 2025b. URL <https://doi.org/10.18653/v1/2025.findings-emnlp.505>.

- Graeme Ritchie. Current directions in computational humour. *Artificial Intelligence Review*, 16(2):119–135, 2001. URL <https://doi.org/10.1023/A:1011610210506>.
- Ryo Sakabe, Hwiyeon Kim, Tatsuya Hirasawa, and Mamoru Komachi. Assessing the capabilities of LLMs in humor: A multi-dimensional analysis of Oogiri generation and evaluation, 2025. URL <https://arxiv.org/abs/2511.09133>. arXiv:2511.09133.
- Alexey Tikhonov and Pavel Shtykovskiy. Humor mechanics: Advancing humor generation with multistep reasoning, 2024. URL <https://arxiv.org/abs/2405.10073>. arXiv:2405.10073.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2022. URL <https://doi.org/10.48550/arXiv.2201.11903>. arXiv:2201.11903.
- Orion Weller and Kevin Seppi. The rJokes dataset: A large scale humor collection. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 6136–6141, 2020. URL <https://aclanthology.org/2020.lrec-1.753/>.
- Tim Winters and Sam Van der Stockt. Evaluating humor generation in an improvisational comedy setting. *Computational Linguistics in the Netherlands Journal*, 14:505–523, 2025. URL <https://www.clinjournal.org/clinj/article/view/214>.
- Zhen Xu, Shaojie Yuan, Li Chen, and Diyi Yang. “a good pun is its own reword”: Can large language models understand puns? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11766–11782, 2024. URL <https://doi.org/10.18653/v1/2024.emnlp-main.657>.
- Tianyu Yu, Bin Ji, Shun Wang, Shunyu Yao, Zhijian Wang, Ganqu Cui, Luqi Yuan, Ning Ding, Yong Yao, Zhiyuan Liu, Maosong Sun, and Tat-Seng Chua. RLPR: Extrapolating RLVR to general domains without verifiers, 2025. URL <https://doi.org/10.48550/arXiv.2506.18254>. arXiv:2506.18254.

- Jiawei Zhang, Sizhe Luo, Ruijie Zhang, and Qi Su. HUMORCHAIN: Theory-guided multi-stage reasoning for interpretable multimodal humor generation, 2025. URL <https://doi.org/10.48550/arXiv.2511.21732>. arXiv:2511.21732.
- Jie Zhang, Lily Jain, Yichi Guo, Justin Chen, Kexin Linda Zhou, Sreeranjani Suresh, Amanda Wagenmaker, Scott Sievert, Taylor Rogers, Kevin Jamieson, Robert Mankoff, and Robert Nowak. Humor in AI: Massive scale crowd-sourced preferences and benchmarks for cartoon captioning, 2024. URL <https://arxiv.org/abs/2406.10522>. Advances in Neural Information Processing Systems, Datasets and Benchmarks Track.
- Shaoxiong Zhong, Zicheng Huang, Shufan Gao, Wen Wen, Lin Lin, Marinka Zitnik, and Pan Zhou. Let’s think outside the box: Exploring leap-of-thought in large language models with creative humor generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13246–13257, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Zhong_Lets_Think_Outside_the_Box_Exploring_Leap-of-Thought_in_Large_Language_CVPR_2024_paper.html.
- Xingcheng Zhou, Zihan Liu, Ariel Sims, Han Wang, Tianyu Pang, Chao Li, Li Wang, Min Lin, and Chao Du. Reinforcing general reasoning without verifiers, 2025. URL <https://arxiv.org/abs/2505.21493>. arXiv:2505.21493.